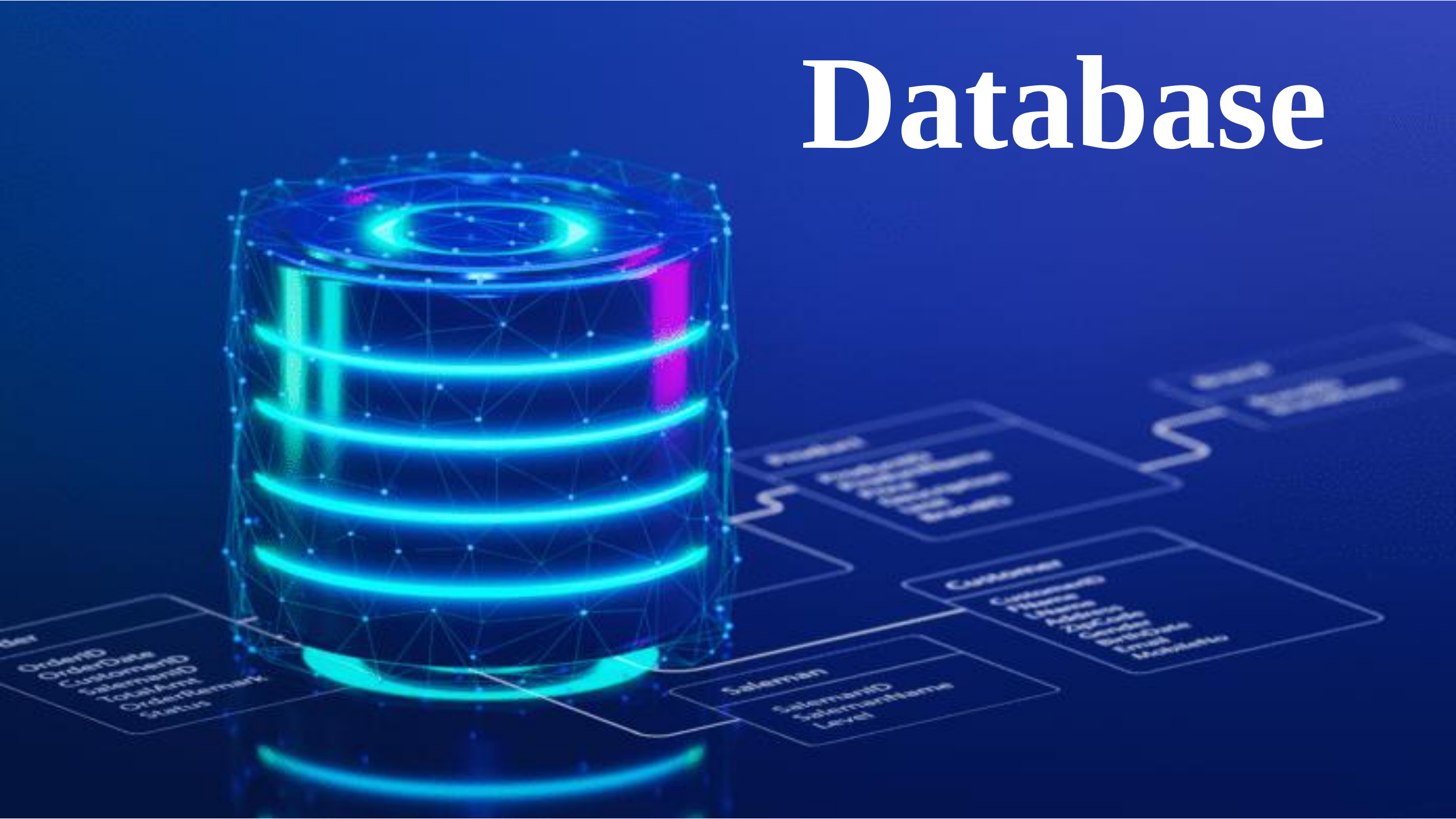
Introduction to Database Management Systems

Topics

- The Need for Databases
- Data Models
- Relational Databases
- Database Design
- Storage Manager
- Query Processing
- Transaction Manager


Data

Database



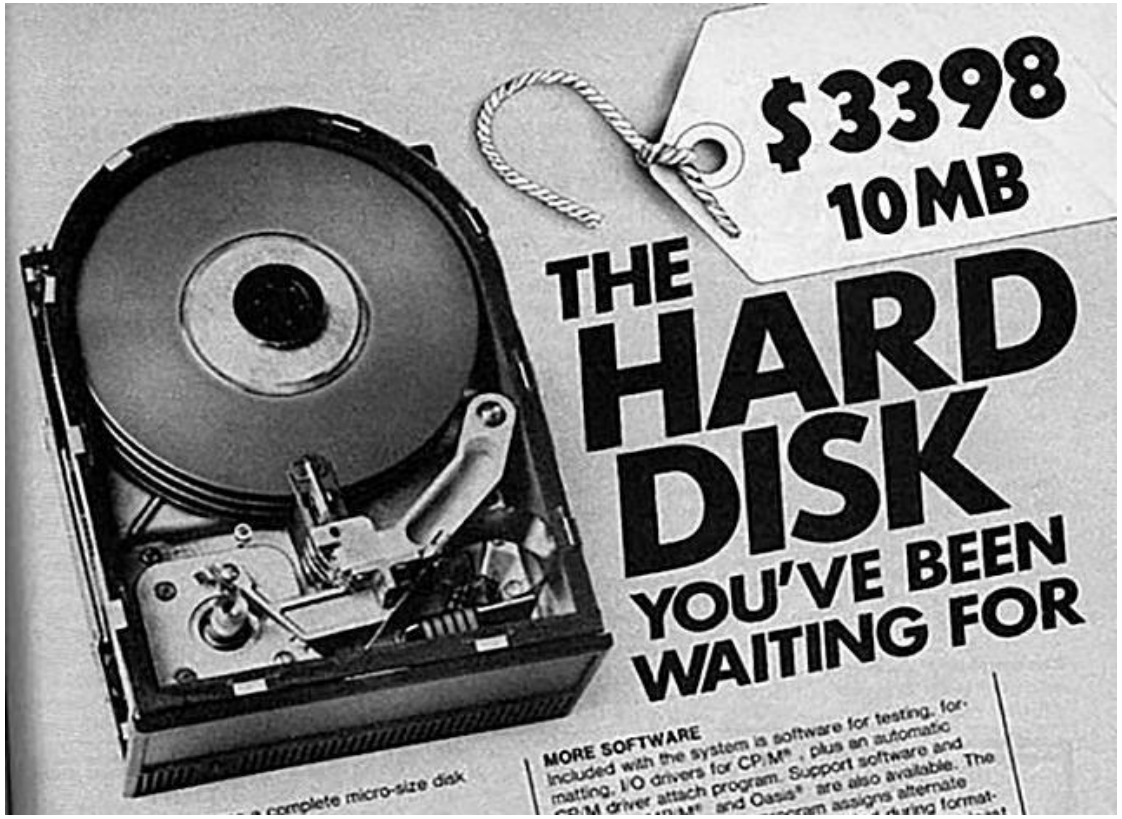
Database Management System

History of DBMS

- 1950s and early 1960s:
  - Data processing using **magnetic tapes** for storage
    - Tapes provided only **sequential access**
  - **Punched cards** for input

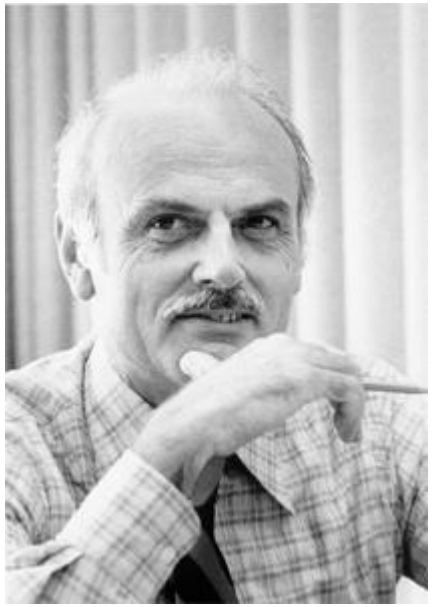
History of DBMS

- Late 1960s and 1970s:
  - **Hard disks: direct access** to data
  - Network and hierarchical data models in widespread use



History of DBMS

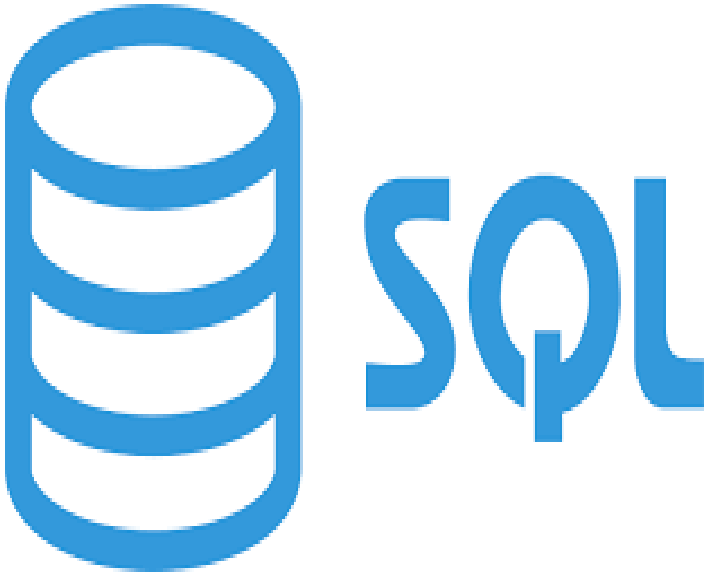
- **Ted Codd**
  - Defined the relational data model
  - Won the Turing Award for this work (1981).
  - IBM Research begins System **R** prototype.
    - The first implementation of **SQL**.



19 August 1923 – 18 April 2003

History of DBMS

- 1980s:
  - **Research relational prototypes** evolve into commercial systems
    - SQL becomes industrial standard
  - **Parallel and distributed** database systems
  - **Object-oriented DBMS**; data representation in the form of objects

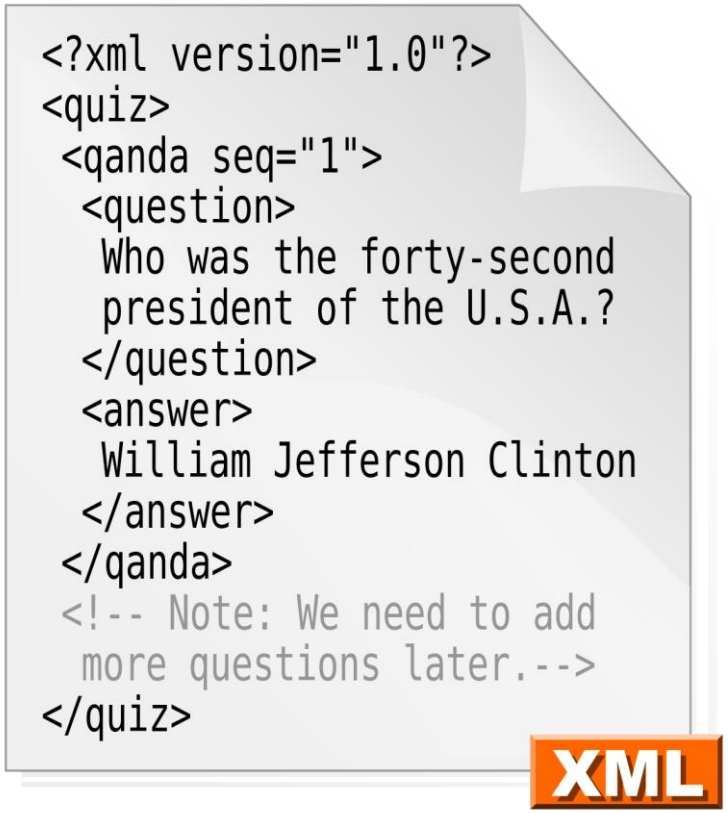


History of DBMS

- 1990s:
  - Large decision support and **data-mining** applications
  - Large multi-terabyte **data warehouses**
  - Emergence of **Web commerce**

History of DBMS

- Early 2000s:
  - Extensible Markup Language (**XML**) and **XQuery** standards
    - A human- and machine-readable format.
  - **Automated database administration**



History of DBMS

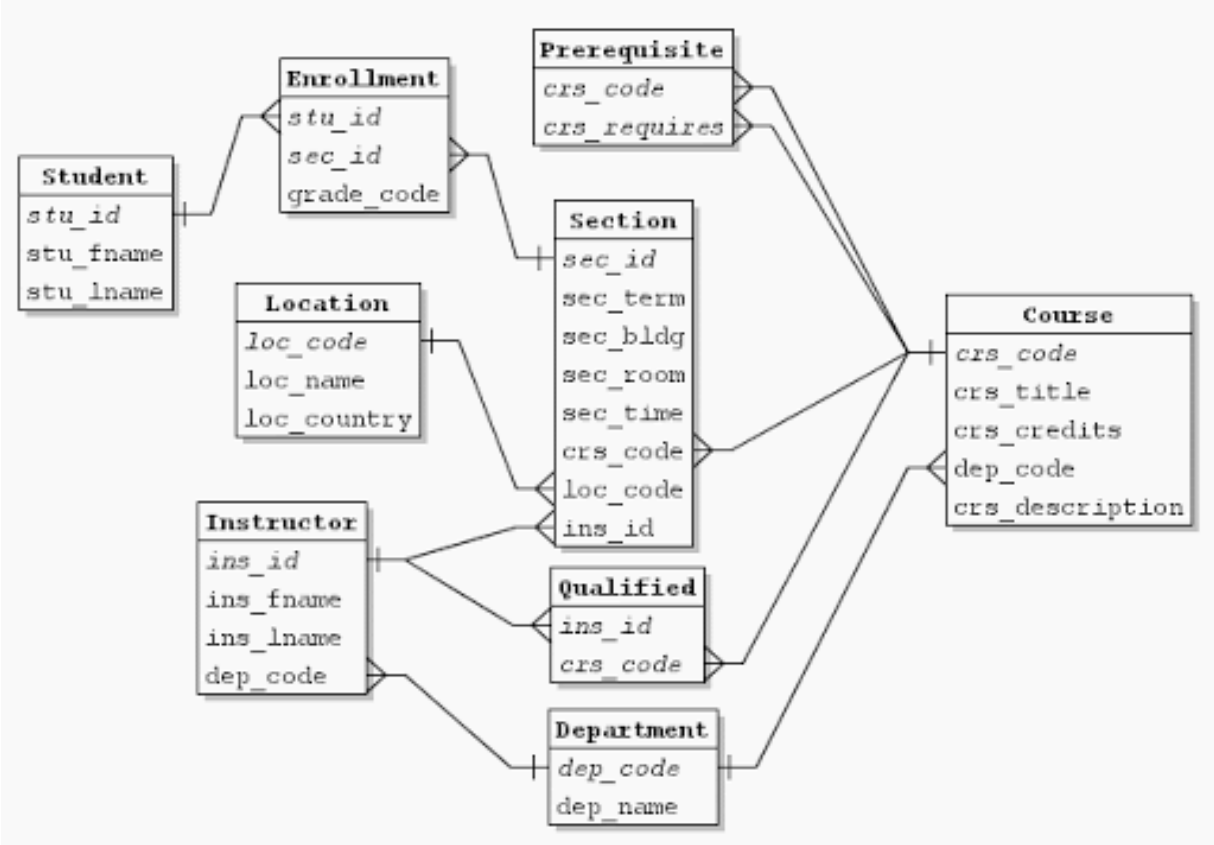
- Later 2000s:
  - Giant data storage systems

DBMS

- DBMS:
  - Contains information about a particular enterprise
  - **Collection** of interrelated data
  - Set of programs to **access** the data
  - **Convenient** and **efficient** environment to use
- **Database Applications**:
  - **Banking**: transactions
  - **Airlines**: reservations, schedules
  - **Universities**: registration, grades
  - **Sales**: customers, products, purchases
  - **Online retailers**: order tracking, customized recommendations
  - **Manufacturing**: production, inventory, orders, supply chain
  - **Human resources**: employee records, salaries, tax deductions
  - …

DBMS

- Examples
  - Add new students, instructors, and courses
  - Register students for courses, and generate class rosters
  - Assign grades to students, compute GPA
  - Generate transcripts

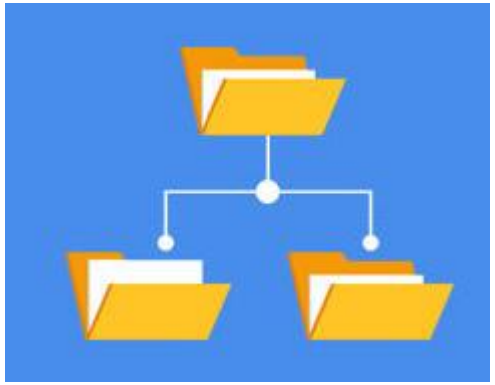


File System

- **File system** or **filesystem:**
  - Used to control how data is stored and retrieved.

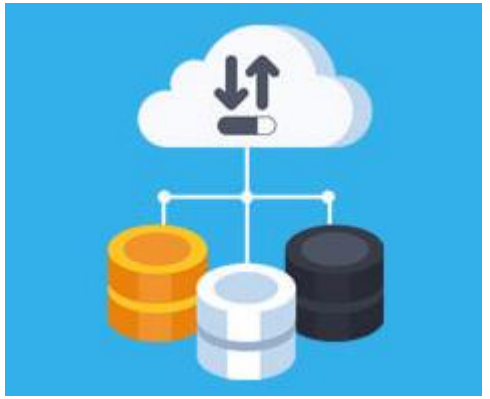
Filesystem

More unstructured data store for storing arbitrary, probably unrelated data.



Database

Used for storing related, structured data, with well defined data formats, in an efficient manner for insert, update and/or retrieval (depending on application).



File systems

Drawbacks:

- **Data redundancy and inconsistency**
  - Multiple file formats, duplication of information in different files
- **Difficulty in accessing data**
  - Need to write a new program to carry out each new task
- **Data isolation**
  - Multiple files and formats, no manifest…
- **Integrity problems**
  - Integrity constraints (e.g., account balance > 0) become "buried" in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones

File systems

Drawbacks:

- **Atomicity of updates**
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: Transfer of funds from one account to another should either complete or not happen at all.
- **Concurrent access by multiple users**
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- **Security problems**
  - Hard to provide user access to some, but not all, data

Levels of Abstraction

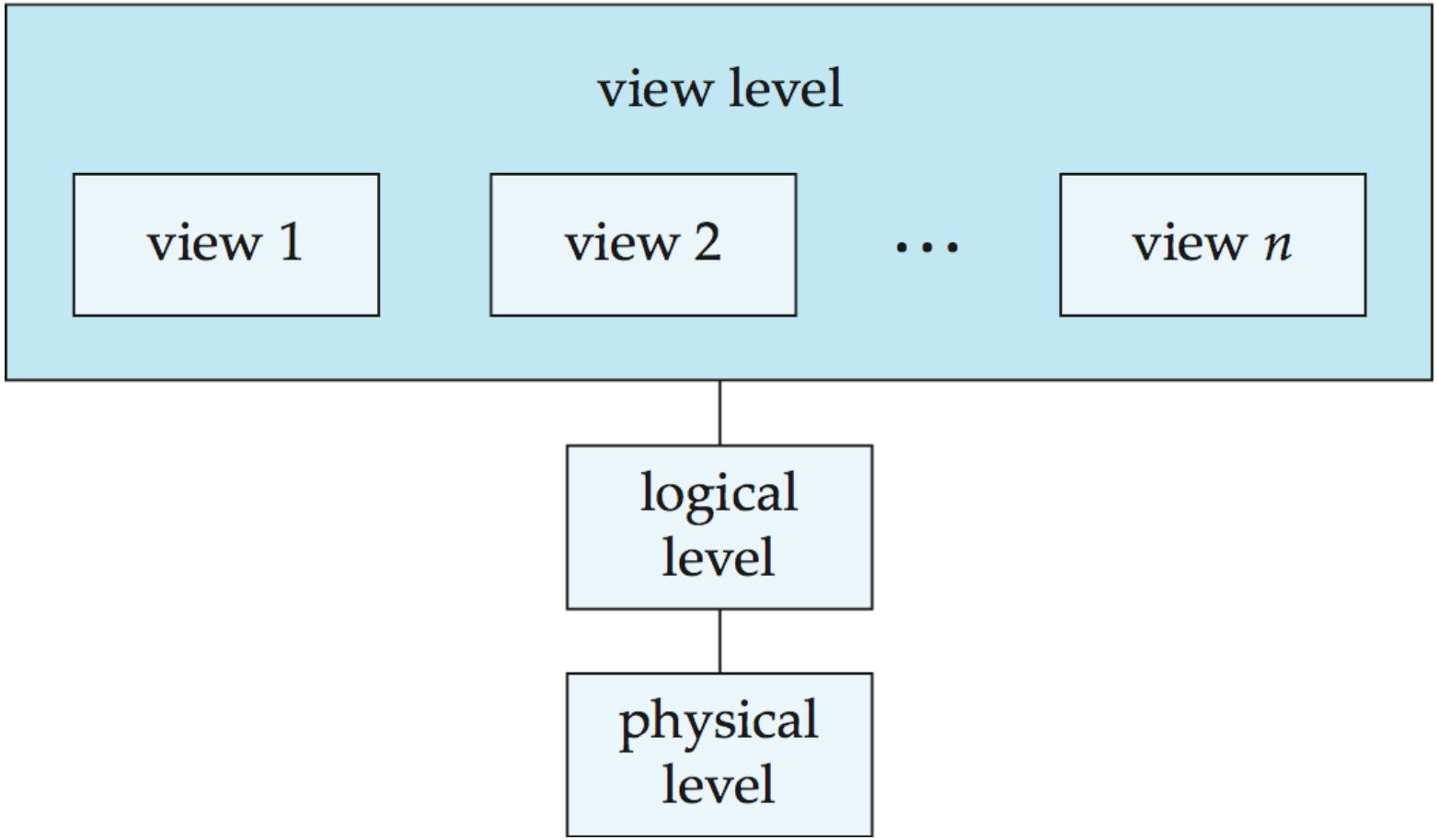
- **Physical level:** describes how a record (e.g., instructor) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

```
type instructor = record
        ID : string;
        name : string;
        dept_name : string;
        salary : integer;
end;
```

- **View level:** application programs hide details of data types.
  - Views can also hide information (such as an employee's salary) for security purposes.

View of Data

- DBMS architecture



Instances and Schemas

- **Logical Schema**
  - The overall logical structure of the database
- **Physical schema**
  - The overall physical structure of the database
- **Instance**
  - The actual content of the database at a particular point in time
- **Physical Data Independence**
  - The ability to modify the physical schema without changing the logical schema

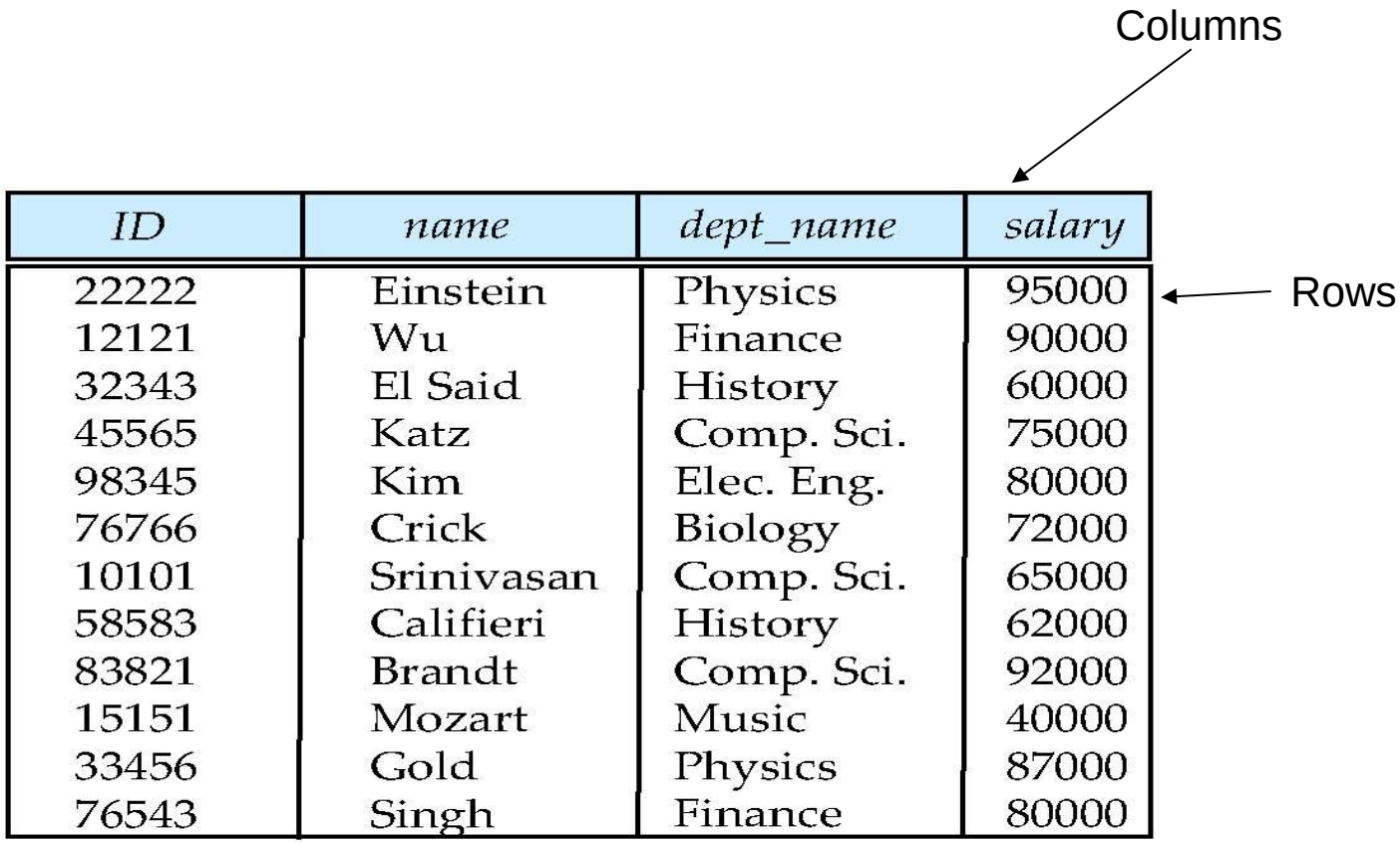
Data Models

- A collection of tools for describing
  - Data
  - Data relationships
  - Data semantics
  - Data constraints

- **Types**:
  - **Hierarchical Model**
  - **Network Model**
  - **Relational model**
  - **Entity-Relationship data model**
  - **Object-based data models**
  - **Semi-structured data model**

Relational Model

- Stores all the data in various tables.



ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Data Definition Language (DDL)

- Specific notation for defining the database schema

  Example:

```
create table instructor (
        ID          char(5),
        name        varchar(20),
        dept_name   varchar(20),
        salary      numeric(8,2))
```

- **DDL Compiler**: generates a set of table templates stored in a **data dictionary**
- **Data Dictionary**: contains metadata (i.e., **data about data**)
  - Database schema
  - Integrity constraints
    - Primary key (ID uniquely identifies instructors)
  - Authorization
    - Who can access what

Data Manipulation Language (DML)

- Language for accessing and manipulating the data organized by the appropriate data model
  - a.k.a. query language

- DML language classes:
  - **Pure**
    - Used for proving properties about computational power and for optimization
      - Relational Algebra
      - Tuple relational calculus
      - Domain relational calculus
  - **Commercial**
    - Used in commercial systems
      - Structured Query Language (SQL)

SQL

- The most widely used commercial language
- SQL is NOT a Turing machine equivalent language
- Usually embedded in some higher-level language
  - To be able to compute complex functions
- Application programs generally access databases through one of
  - Language extensions to allow embedded SQL
  - Application program interface (e.g., ODBC/JDBC)
    - Allow SQL queries to be sent to a database

Database Design

- The process of designing the general structure of the database:
- **Types**:
  - **Logical Design**
    - Deciding on the database schema.
    - Requires finding a “good” collection of relation schemas.
    - **Business decision:**
      - Attributes to be recorded in the database
    - **Computer Science decision:**
      - Relation schemas
      - Distribution of the attributes among various relation schemas
  - **Physical Design**
    - Deciding on the physical layout of the database

Database Design

- Is there any problem with this relation?

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Design Approaches

- Need to come up with a methodology to ensure that each of the relations in the database is “good”!

- Methods:
  - **Entity Relationship Model**
    - Models an enterprise as a collection of *entities* and *relationships*
    - Represented diagrammatically by an *entity-relationship diagram:*
  - **Normalization Theory**
    - Formalize what designs are bad, and test for them

Object-Relational Data Models

- Relational model: flat, “atomic” values
- Object Relational Data Models
  - Including object orientation and constructs to deal with added data types.
  - Allow attributes of tuples to have complex types,
    - Including non-atomic values such as nested relations.
  - Preserve relational foundations while extending modeling power.
    - In particular the declarative access to data,
  - Provide upward compatibility with existing relational languages.

Extensible Markup Language (XML)

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange **data**, not just documents
- Become the basis for all new generation data interchange formats.

```
▼<note>
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
  </note>
```

Database Engine

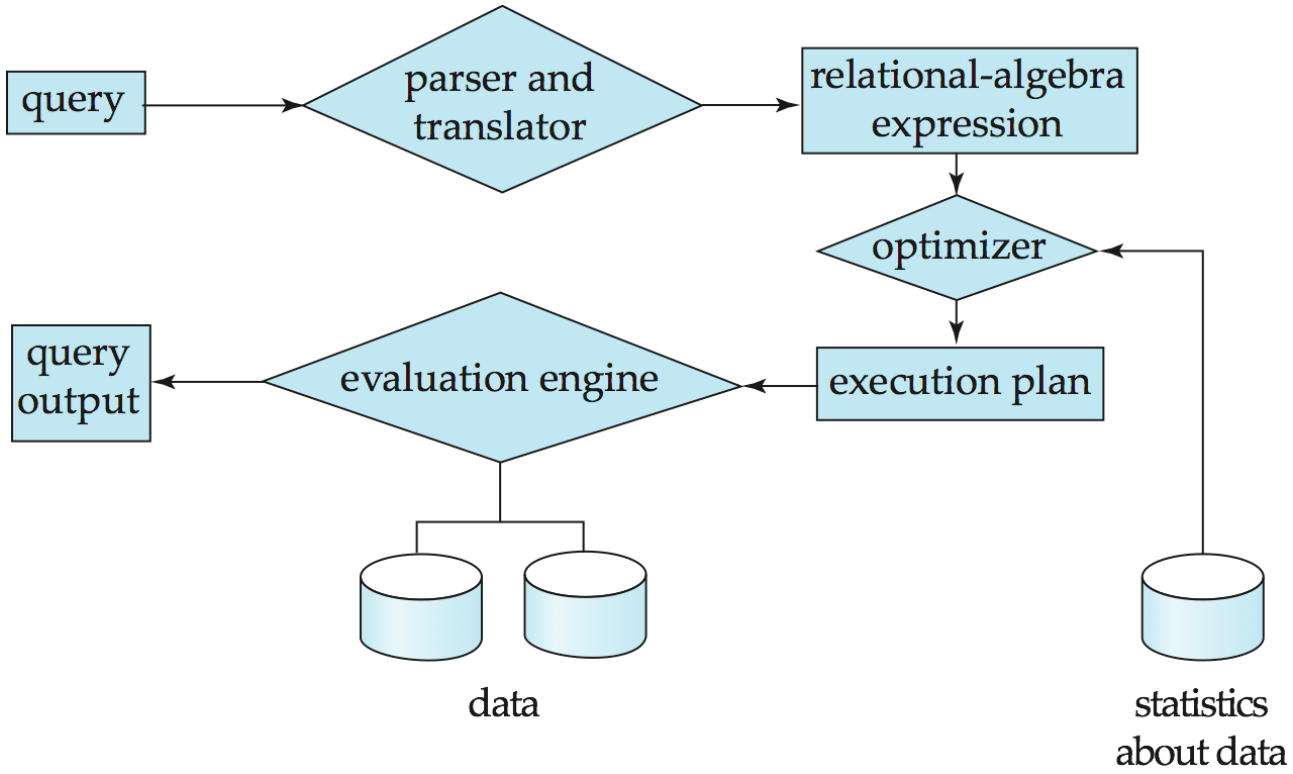
- Storage manager
- Query processing
- Transaction manager

Storage Management

- **Storage manager:**
  - A program module
  - Provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- Tasks:
  - Interaction with the OS file manager
  - Efficient storing, retrieving and updating of data
- Issues:
  - Storage access
  - File organization
  - Indexing and hashing

Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



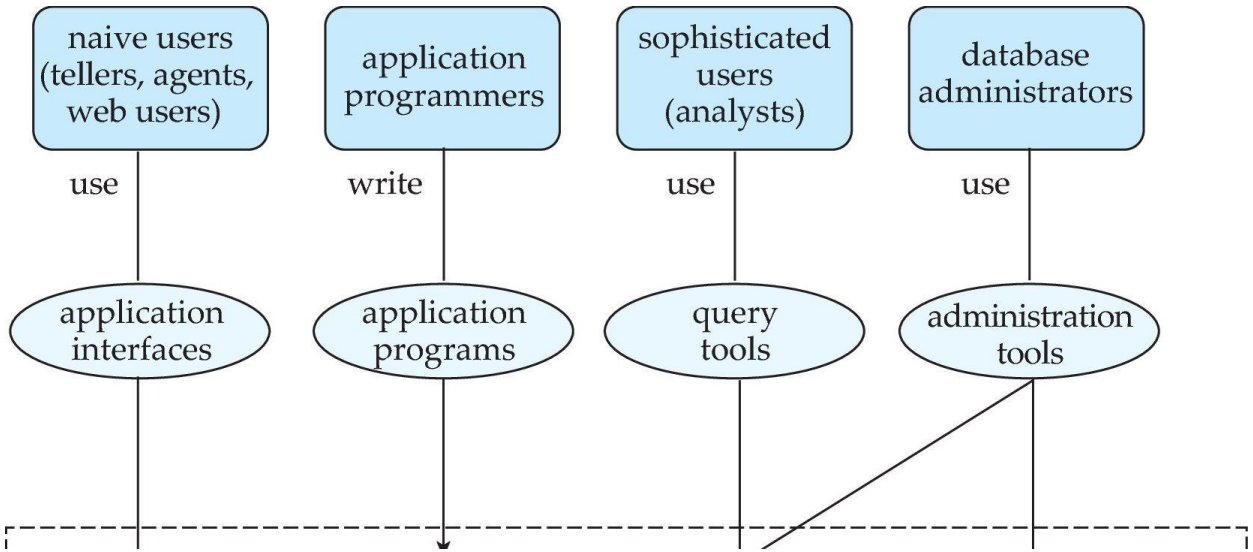
Query Processing

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- Cost difference between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
  - Depends critically on statistical information about relations which the database must maintain
  - Need to estimate statistics for intermediate results to compute cost of complex expressions

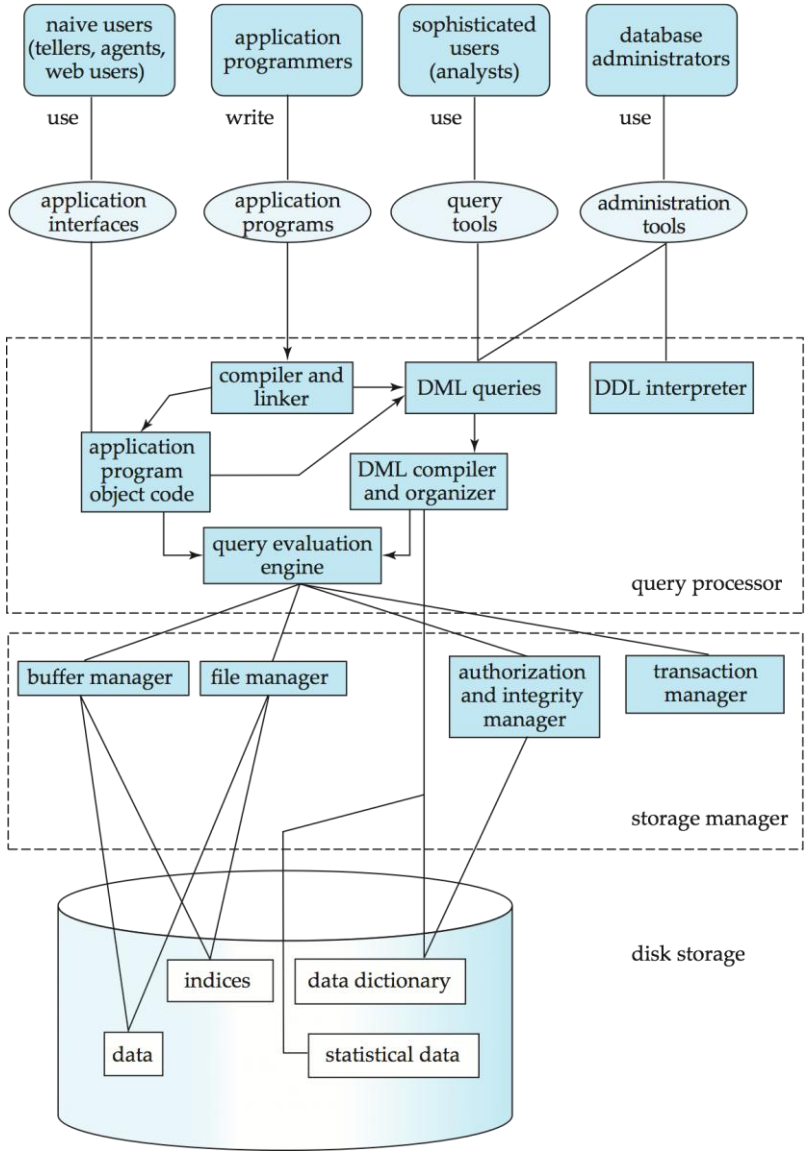
Transaction Management

- What if the system fails?
- What if more than one user is concurrently updating the same data?
- **Transaction**
  - A collection of operations that performs a single logical function in a database application
- **Transaction-management component**
  - Ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- **Concurrency-control manager**
  - Controls the interaction among the concurrent transactions, to ensure the consistency of the database.

Database Users and Administrators



Database System Internals



Database Architecture

- Influenced by the underlying computer system on which the database is running.
- Types:
  - **Centralized**
  - **Client-server**
  - **Parallel (multi-processor)**
  - **Distributed**

Conclusion

- Databases organize and manage data
- DBMS evolved over decades
- Critical for modern applications

piran.mj2@gmail.com